

Architectural Design

Architectural design is the **first stage of the software design process**.

It focuses on **how a software system should be organized** and defines the **overall structure** of that system.

It identifies the **main structural components (modules)** of the system and the **relationships or interactions** between them.

In short, it acts as a **bridge between requirements engineering and detailed design**.

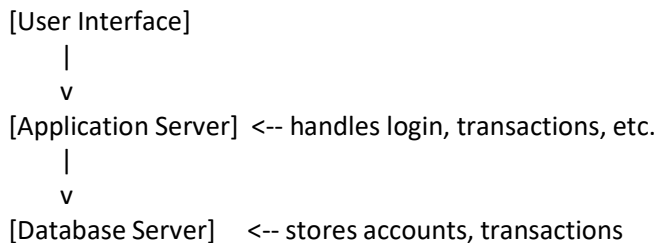
Example: Online Banking System

Let's say you are designing an **Online Banking System**.

Architectural Design Decisions:

- Divide the system into **modules** like:
 - Login & Authentication
 - Account Management
 - Transaction Processing
 - Report Generation
 - Database
- Decide how these modules **communicate** (e.g., via APIs).
- Choose **architecture style** (e.g., **Client-Server Architecture**):
 - Client → User's web browser or app
 - Server → Handles logic and database

Simple visualization:



Architectural Design Decisions:

During this process, software architects must make several important **structural and organizational decisions**, such as:

1. **Use of Generic Architecture:**
 - Decide whether an existing architecture (like client-server, layered, MVC, etc.) can be reused.
2. **System Structuring Approach:**
 - Determine how the system will be divided into subsystems or layers.
3. **Architectural Styles or Patterns:**
 - Choose appropriate styles such as layered, repository, client-server, or pipe-and-filter.
4. **Distribution Strategy:**
 - Decide how system components will be distributed across hardware or network systems.
5. **Documentation Method:**
 - Specify how the architecture will be represented — diagrams, design documents, UML, etc.
6. **Evaluation Method:**
 - Determine how the architecture will be analyzed for performance, scalability, and security before implementation.

Impact of Architectural Design on System Quality:

The chosen architecture **directly affects** the key **non-functional requirements** of the system:

Non-Functional Requirement Architectural Influence

Maintainability	Use of modular or layered architecture makes updates and debugging easier.
Performance	Efficient data flow and component distribution improve response time.
Availability	Redundant components or fault-tolerant architecture ensure uptime.
Security	Segregation of components and secure communication channels protect data.
Safety	Safe component interaction and fault isolation prevent system hazards.

Four Essential Architectural Views:

An **architectural view** is a **specific way of looking at a software system's architecture** — from a particular **perspective or concern**.

1. Logical View

Focus:

How the system is structured in terms of **main functions, classes, and relationships**.

Used by:

End users, analysts, and designers.

Purpose:

To understand **what the system does**.

Example:

In an online shopping system:

- Classes → Customer, Order, Product, Payment.
- Relationships → A customer *places* an order, an order *contains* products.

2. Development View (Implementation View)

Focus:

How the software is **organized for development** — i.e., how the code and modules are divided.

Used by:

Developers and software managers.

Purpose:

To plan coding, version control, and module responsibilities.

Example:

- Frontend module (UI)
- Backend module (API)
- Database access module

3. Process View (Dynamic View)

Focus:

How the system's **processes interact at runtime** — showing concurrency, communication, and performance.

Used by:

System integrators and testers.

Purpose:

To check **system performance, reliability, and communication** between parts.

Example:

When a user places an order:

- The frontend sends a request → backend processes it → database stores the record.

4. Physical View (Deployment View)

Focus:

How software components are **distributed across hardware** (servers, devices, networks).

Used by:

System engineers and network designers.

Purpose:

To plan **deployment and system distribution**.

Example:

- Web app hosted on AWS server

- Database on a separate server
- Backup system on cloud storage

Architectural Patterns

An **Architectural Pattern** is a **general, reusable solution** to a commonly occurring problem in software architecture. It represents **proven design practices** that describe how software systems should be structured and how their components interact.

These patterns are not concrete designs, but **high-level templates** that guide the system's organization.

In short:

Architectural patterns capture good design experience and help in presenting, sharing, and reusing architectural knowledge.

Purpose:

- To **standardize system organization** for common problems.
- To **improve reusability** and **maintainability**.
- To **reduce design risk** by applying well-tested structures.

Commonly Used Architectural Patterns:

Some widely used patterns that capture good design principles are:

1. **Model–View–Controller (MVC)**
2. **Layered Architecture**
3. **Client–Server Architecture**
4. **Repository Pattern**
5. **Pipe and Filter Pattern**

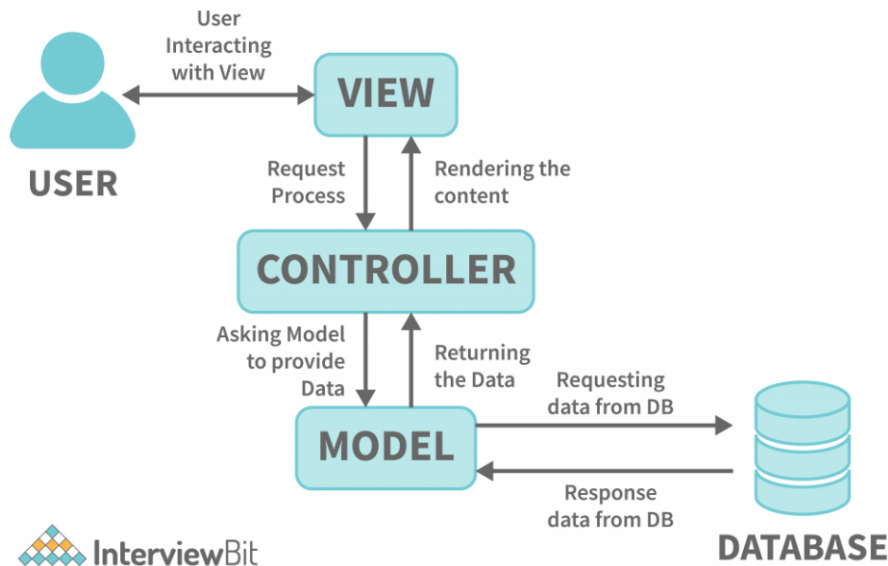
1. Model–View–Controller (MVC) Pattern

The **MVC Pattern** is one of the most common architectural patterns used in **web and desktop applications**.

It separates the **data (Model)**, **presentation (View)**, and **interaction (Controller)** into different components to improve modularity and flexibility.

Components:

Component	Function
Model	Manages system data and business logic. Encapsulates the application state and defines operations on data.
View	Defines and manages how data is presented to the user (UI). Displays information received from the Model.
Controller	Handles user inputs (keyboard, mouse, button clicks), processes them, and updates the Model or View accordingly.



Example (Online Shopping System):

- **Model:** Product, Customer, Order, Payment data.
- **View:** Web pages showing product lists, shopping cart, and order confirmation.
- **Controller:** Handles user actions like “Add to Cart” or “Checkout” and updates Model and View accordingly.

Flow of Control in MVC (online shopping)

1. **User interacts with View**
→ clicks “Add to Cart” on the product page.
2. **Controller receives the request**
→ identifies which product was clicked and sends instruction to Model.
3. **Model updates data**
→ adds the product to user’s cart in the database.
4. **Model notifies Controller/View of changes**
→ “Cart updated.”
5. **View refreshes and displays updated data**
→ shows new cart total and product count.

Advantages:

- **Separation of concerns:** The system is divided into three layers (data, UI, logic), making it easier to manage.
- **Flexibility:** You can change the user interface (View) without affecting the data (Model).
- **Multiple views:** The same data can be presented in different ways (e.g., chart, table).
- **Reusability:** Components can be reused across different applications.

Disadvantages:

- **Increased complexity:** For small or simple systems, MVC may introduce unnecessary code.
- **Communication overhead:** More interactions among components can make debugging harder.

2. Layered Architecture Pattern

Layered architecture organizes software into layers, each responsible for a specific type of work. A layer can only interact with the layer directly below it.

Layers and Detailed Example: Online Shopping System

Layer	Responsibility	Detailed Example
Presentation Layer (UI Layer)	Displays information to the user and captures user input.	- Web page displays product list, cart, checkout form. - Mobile app shows product images, reviews, and

Authentication & Authorization Layer	Verifies user identity and access permissions.	buttons to add/remove items from cart. - Login/signup validation. - Ensures that only registered users can place orders. - Admin can manage products but customers cannot.
Business Logic Layer (Application Layer)	Processes user requests and performs business rules.	- When a user adds an item to cart → calculates total price including tax and discount. - Checks stock availability. - Handles promotions and coupon codes.
Data Layer (Persistence Layer)	Stores and retrieves data from databases.	- Stores product catalog, user accounts, order history. - Updates inventory when an order is placed. - Retrieves user addresses, payment methods.

Flow Example (Buying a Product):

1. User logs in → Authentication Layer validates credentials.
2. User selects a product → Presentation Layer sends request to Business Logic Layer.
3. Business Logic Layer checks inventory in Data Layer.
4. Business Logic calculates total price, applies discount → returns data to UI Layer.
5. User confirms order → Business Logic updates order details in Data Layer.
6. UI Layer shows confirmation message.

Advantages:

- Easier to maintain; layers can be modified independently.
- Supports code reuse.

Disadvantages:

- Can be slower due to multiple layers of processing.

3. Repository Architecture Pattern

In repository architecture, multiple components share data through a central repository. Components never interact directly; all communication is through the repository.

Example: Online Shopping System Components

Component	Role
Product Management	Adds, updates, or removes products from the repository.
Order Management	Reads product info from repository, creates orders, updates order status.
Payment System	Reads order data, processes payments, updates payment status.
Customer Support	Reads order and customer data for handling complaints.

Central Repository:

- Stores all system data: product catalog, user accounts, order history, payment history.
- All components read/write to this central data store.

Flow Example (Order Processing):

1. Customer places an order → Order Management writes order to Repository.
2. Payment System reads the order from Repository → Processes payment → Updates status in Repository.
3. Inventory component reads the order → Updates stock quantity in Repository.
4. Customer Support reads order and payment info from Repository → Assists customer if needed.

Advantages:

- Components are independent and reusable.
- Changes in one component automatically reflect for all others via the repository.

Disadvantages:

- Single point of failure: if repository fails, entire system is affected.
- Can become a bottleneck if many components access repository simultaneously.

4. Client-Server Architecture Pattern

A **client-server system** has two main types of components:

- **Clients:** Programs that request services (e.g., a web browser, mobile app).
- **Servers:** Programs that provide services (e.g., web server, database server).

Key Points:

- Multiple clients can connect to the same server.
- Communication happens over a **network** (like the internet).
- Servers provide centralized resources or services.

Example: Film Library System

- Clients: Your laptop, phone, or tablet accessing the library.
- Servers:
 - Catalog server → stores film details
 - Video server → streams films
 - Picture server → stores posters/images
 - Web server → hosts the website

Flow:

Client (Laptop) → Web server → Catalog server (gets film info)

Client (Phone) → Video server (streams movie)

Advantages:

- Centralized control of data and services.
- Multiple clients can use the same server resources.

Disadvantages:

- Server is a **single point of failure**. If it goes down, clients can't access services.

5. Pipe and Filter Architecture Pattern

A **pipe and filter system** is designed so that **data flows through a sequence of processing steps (filters)**. Each step performs a single transformation and passes it to the next step through a "pipe."

Key Points:

- Each filter does **one specific job**.
- Filters are independent and can be reused.
- Pipes carry data from one filter to the next.

Example: Online Shopping Order Processing

Filters (steps):

1. **Read Order** → Filter collects order details from the customer (items, quantity, customer info).
2. **Validate Order** → Filter checks if the items are in stock, validates payment details, and checks shipping address.
3. **Process Payment** → Filter processes the payment (credit card, UPI, or wallet).
4. **Update Inventory** → Filter reduces stock for the purchased items.
5. **Generate Invoice & Send Confirmation** → Filter creates the invoice and sends confirmation email/SMS to the customer.
6. **Notify Delivery Team** → Filter sends order details to the warehouse/delivery system for shipping.

Flow:

Customer Order → [Read Order] → [Validate Order] → [Process Payment] → [Update Inventory] → [Generate Invoice & Notify] → [Notify Delivery Team]

Advantages:

- Modular and easy to maintain.
- Each filter can be updated independently.

Disadvantages:

- Data may need to be reformatted for different filters.
- Performance depends on sequential processing; can be slower for large datasets.

Application Architecture

The structure, patterns, and techniques used to design and build an application. It defines how an application's components interact and ensures it meets business requirements.

Purpose:

- Makes applications **scalable** and **reliable**.
- Provides a framework for **reuse** and initial design when the developer is unfamiliar with the application type.
- Helps **judge components** and compare similar applications.

Modular Decomposition

- Breaking down a system into smaller, manageable **modules**.
- **Purpose:**
 - Simplifies design and maintenance.
 - Helps manage complex systems.
- **Two models of decomposition:**
 1. **Object Models:** Decompose system into interacting **objects**.
 2. **Data-Flow Models:** Decompose system into **functional modules** that transform inputs to outputs (also called pipeline model).

Activities in Architectural Design Process

1. **System Structuring:**
 - Divide system into **major subsystems**.
 - Identify **communication mechanisms** between them.
2. **Control Modeling:**
 - Model **control relationships** between subsystems (which subsystem controls or coordinates others).
3. **Modular Decomposition:**
 - Break **subsystems** into lower-level **modules** (components, objects, etc.).